TenantPro: Architecture, Implementation and Current State

Oct 30, 2026 · @Anush Kulal M

A technical report on TenantPro, a two-sided rent-management system built for the Indian residential rental market. It covers the architecture, the features that are actually built, the payment model, the security design, what the tests prove, and what is still missing.

Abstract

TenantPro is a working rent-management system with two sides to it, already deployed and in use. The client is React Native on Expo SDK 54; the server is Node and Express over MySQL. Together they come to 38,251 lines across 123 source files: 14,444 in the backend, 21,609 in the redesigned mobile client and 2,198 in eight test suites. The API exposes 87 endpoints over 22 tables. The client has 28 screens.

The one decision that shapes everything else is that **money never passes through it**. Rent goes straight from tenant to landlord, either by UPI or through a payment gateway wired to the landlord's own merchant account. That keeps the operator outside the RBI's payment-aggregator perimeter, which comes with an authorisation requirement and a ₹25 crore net-worth floor. It also means the platform bills nobody and holds nobody's funds.

An application that does not touch the money also cannot see it, so everything rests on an explicit confirmation step. A payment counts as real only once somebody other than the tenant vouches for it. There are two such vouchers: the landlord tapping Confirm, and the payment gateway answering a server-to-server query. Both run through the same settlement function, which is why they can never disagree about when a rent cycle has cleared.

The current build carries 386 assertions across six suites. Of those, 168 run end-to-end against a live server and a real MySQL instance, and 71 drive the compiled application inside a browser engine. The sections that follow cover the architecture, the feature set as it stands with screenshots as evidence, the payment and session-security models, how the verification was done, and the work that has been identified but not built.

1. Introduction

Most residential tenancy in India is run on paper, in WhatsApp threads and in the landlord's memory. Rent arrives by UPI transfer or in cash, and the record of it is a screenshot. The tenant who paid has no receipt their landlord has acknowledged. The

landlord with six rooms has no dependable way of telling who is behind.

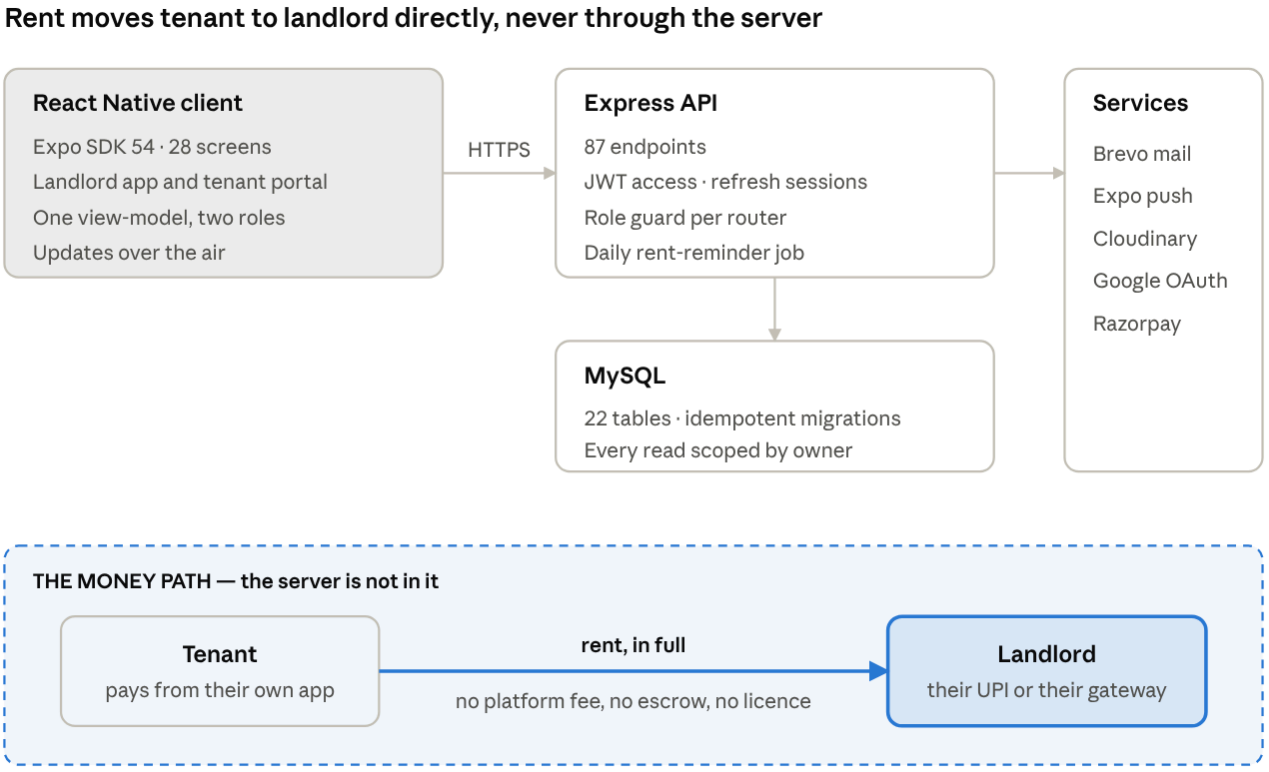
TenantPro works on the day-to-day side of that relationship, not the legal side. A landlord gets a roster of properties, rooms and tenants, with due dates worked out from each tenancy's own anniversary. The tenant gets a view of the same records from the other direction: what they owe, when it is due, what they have already paid, and which of it the landlord has acknowledged.

Three constraints did more to shape the build than any feature request did.

1. **The application cannot see money.** There is no view of a bank account anywhere in it, so it is in no position to assert that a transfer happened. Until something outside the app vouches for it, a payment is only a *claim*. That is a matter of correctness, and no amount of engineering makes it go away.
2. **Collecting rent centrally is a regulated activity.** Pooling tenants' rent in one account and paying it out again is payment aggregation as far as the RBI is concerned. Money is routed tenant-to-landlord for exactly that reason: to stay outside the perimeter.
3. **Native code cannot ship over the air.** Updates reach the client through `expo-updates`, which can swap out JavaScript but not native modules. Any feature that needs a native SDK drags the release cadence of everything around it down with it, so those were turned down in favour of designs that ship like any other change.

2. System architecture

Every piece of information about a tenancy goes through the server. No part of the payment does. In the diagram below, the dashed band is the money path; it touches the tenant and the landlord, and nothing in between.



TenantPro architecture · client, API, data, services, and the money path outside all of them

One React Native codebase serves both roles. State goes through a single pure function, `deriveVm(state, api)`, which turns server data into everything the screens render. That is what makes the interface testable without a device, and one of the suites in Section 7 exercises it directly.

The API is organised as routers, each with its role guard mounted once at the top rather than repeated in every handler. Landlord and tenant accounts sit in separate tables with their own auto-increments, so knowing who somebody is does not settle what they may do. `requireOwner` turns away a tenant token on a landlord route, and tenant handlers scope every query through the `tenant_users` to `tenants` link.

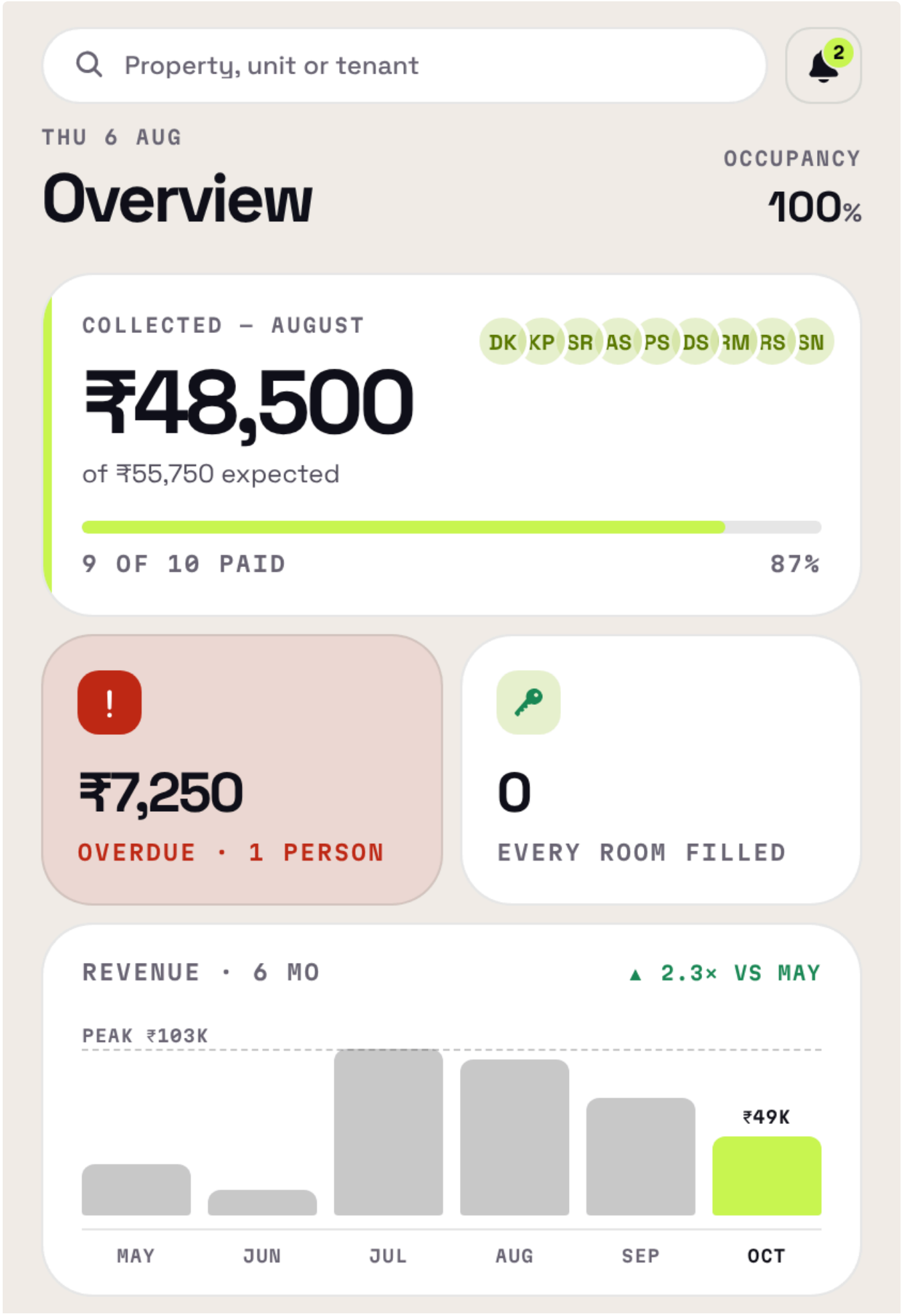
Schema changes apply themselves at boot, and apply safely more than once: `schema.sql` runs `CREATE TABLE IF NOT EXISTS` on every start, and column-level migrations go through `ensureColumn` and `ensureIndex` guards. No deploy has ever needed a manual migration step.

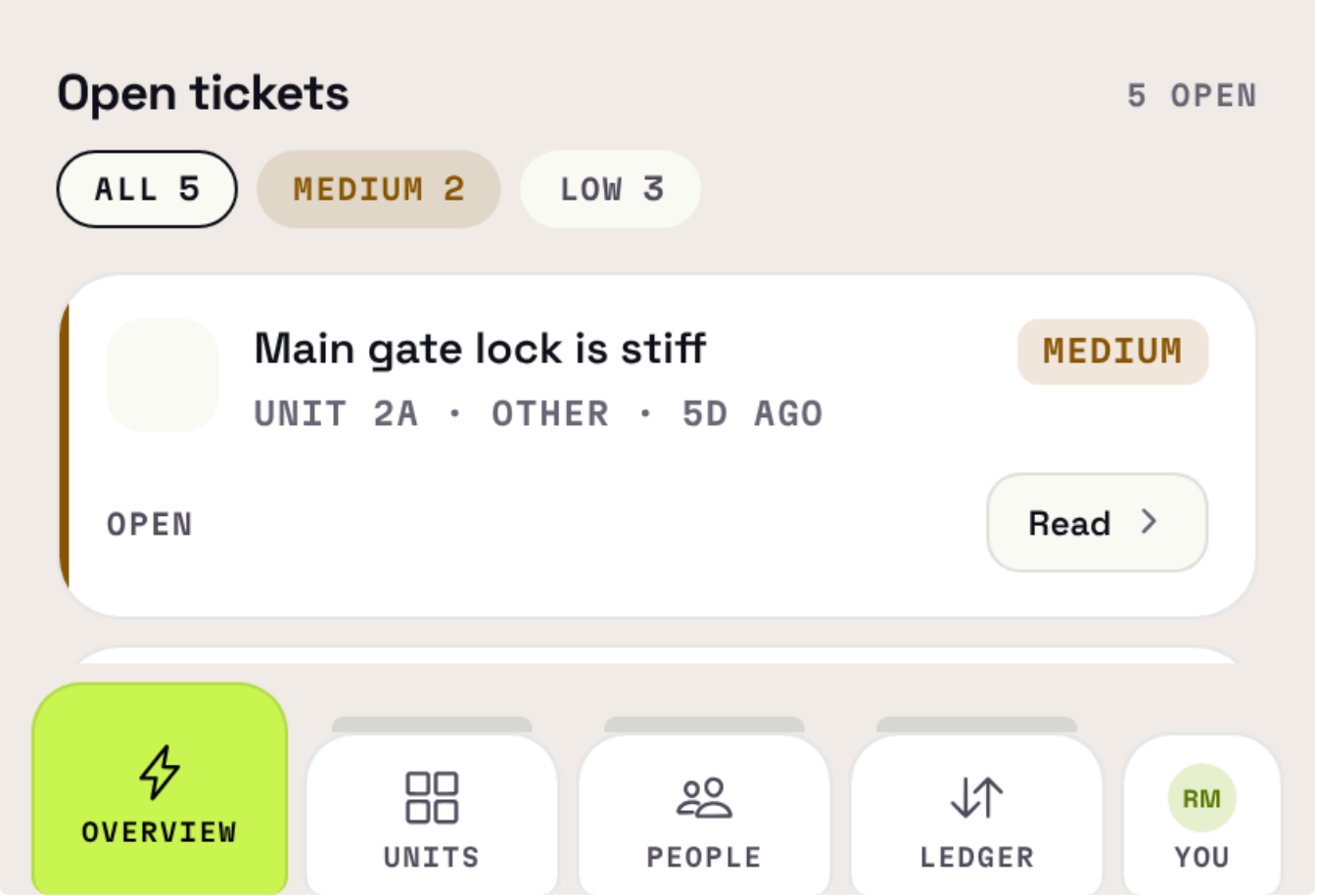
3. Landlord-side implementation

Every screenshot below comes from the running application, working against a seeded database of three properties, six rooms and ten tenants. None of the figures in them are placeholder text.

3.1 Portfolio overview

The landing screen answers the two questions that make a landlord open the app at all: who has paid, and who has not.

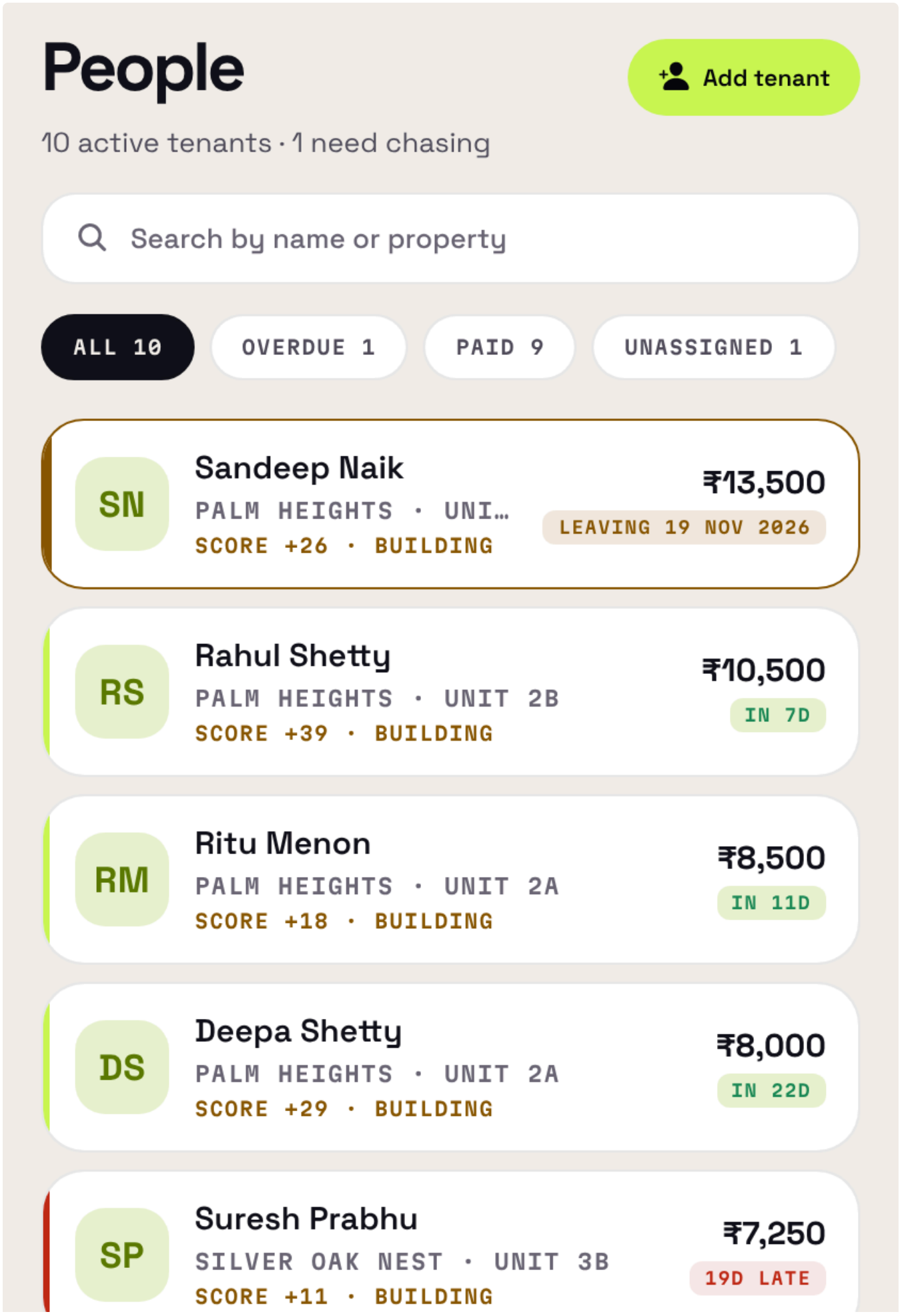

Property, unit or tenant
2
THU 6 AUG
Overview
OCCUPANCY
100%
COLLECTED — AUGUST
DK KP SR AS PS DS RM RS SN
₹48,500
of ₹55,750 expected
9 OF 10 PAID
87%
!
₹7,250
OVERDUE · 1 PERSON
0
EVERY ROOM FILLED
REVENUE · 6 MO
▲ 2.3× VS MAY
PEAK ₹103K
₹49K
MAY
JUN
JUL
AUG
SEP
OCT

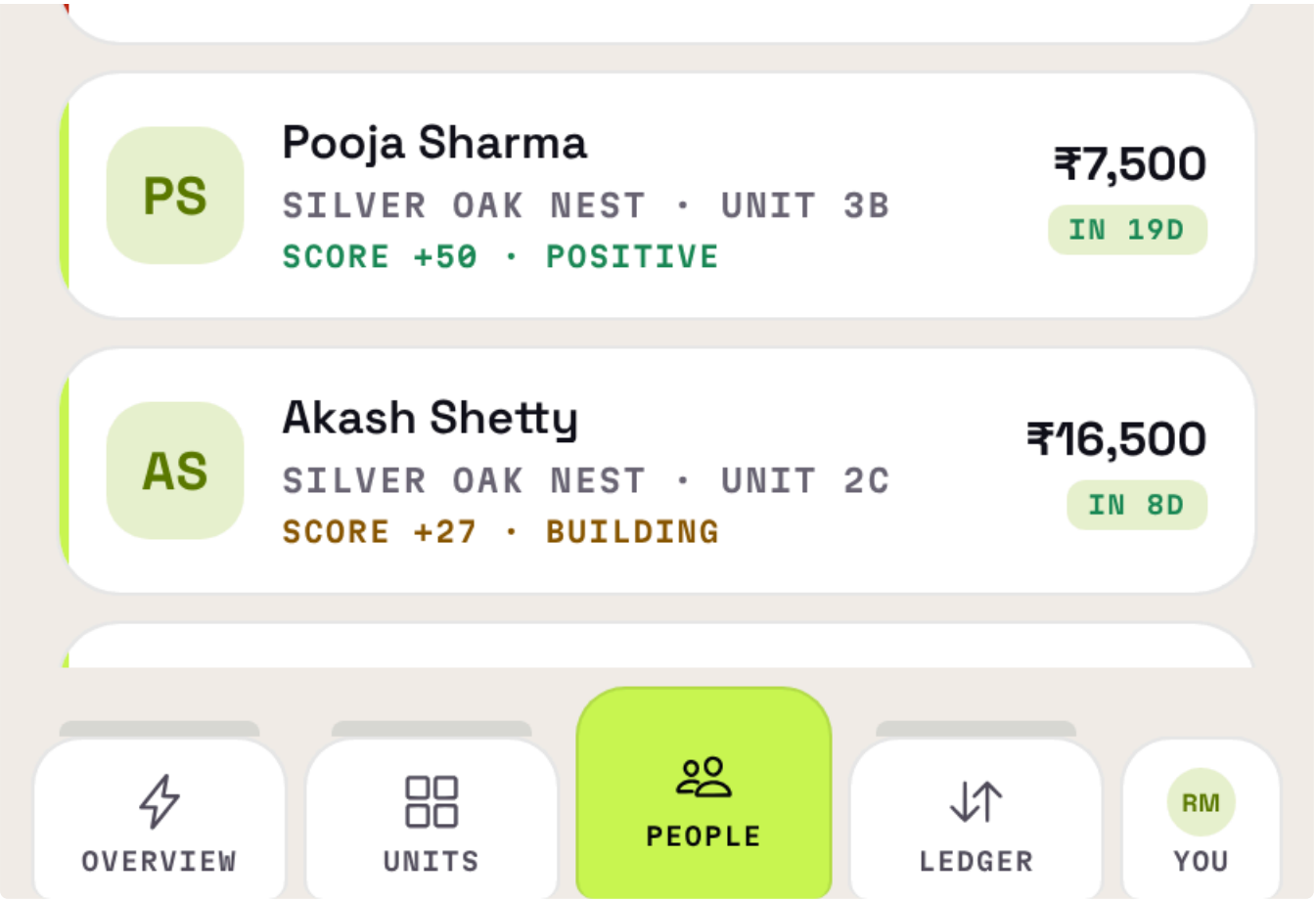


Collection is shown against what was expected rather than as a bare total: ₹48,500 of ₹55,750, nine of ten paid. A number with no denominator is not something anyone can act on. Overdue gets a card of its own, with a count of people attached. The revenue series is built from confirmed payments only. Claims that have been declared but not yet confirmed are left out on purpose, since including them would inflate income on nothing more than a tenant's word.

The bell carries a live count of whatever is waiting on a decision, which means declared payments and join requests.

3.2 The tenant roster

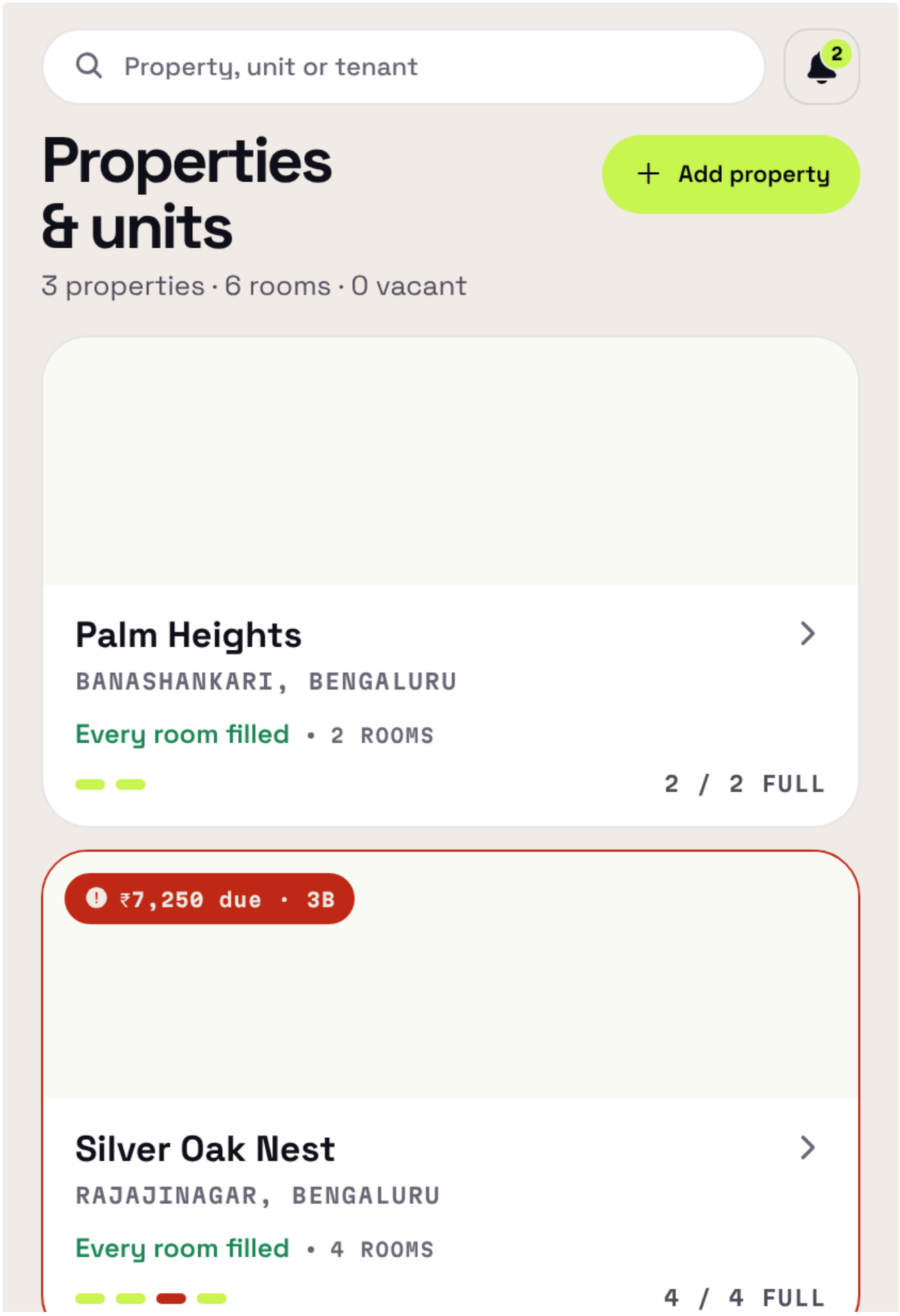

People
Add tenant
10 active tenants · 1 need chasing
Search by name or property
ALL 10
OVERDUE 1
PAID 9
UNASSIGNED 1
SN
Sandeep Naik
PALM HEIGHTS · UNI…
SCORE +26 · BUILDING
₹13,500
LEAVING 19 NOV 2026
RS
Rahul Shetty
PALM HEIGHTS · UNIT 2B
SCORE +39 · BUILDING
₹10,500
IN 7D
RM
Ritu Menon
PALM HEIGHTS · UNIT 2A
SCORE +18 · BUILDING
₹8,500
IN 11D
DS
Deepa Shetty
PALM HEIGHTS · UNIT 2A
SCORE +29 · BUILDING
₹8,000
IN 22D
SP
Suresh Prabhu
SILVER OAK NEST · UNIT 3B
SCORE +11 · BUILDING
₹7,250
19D LATE

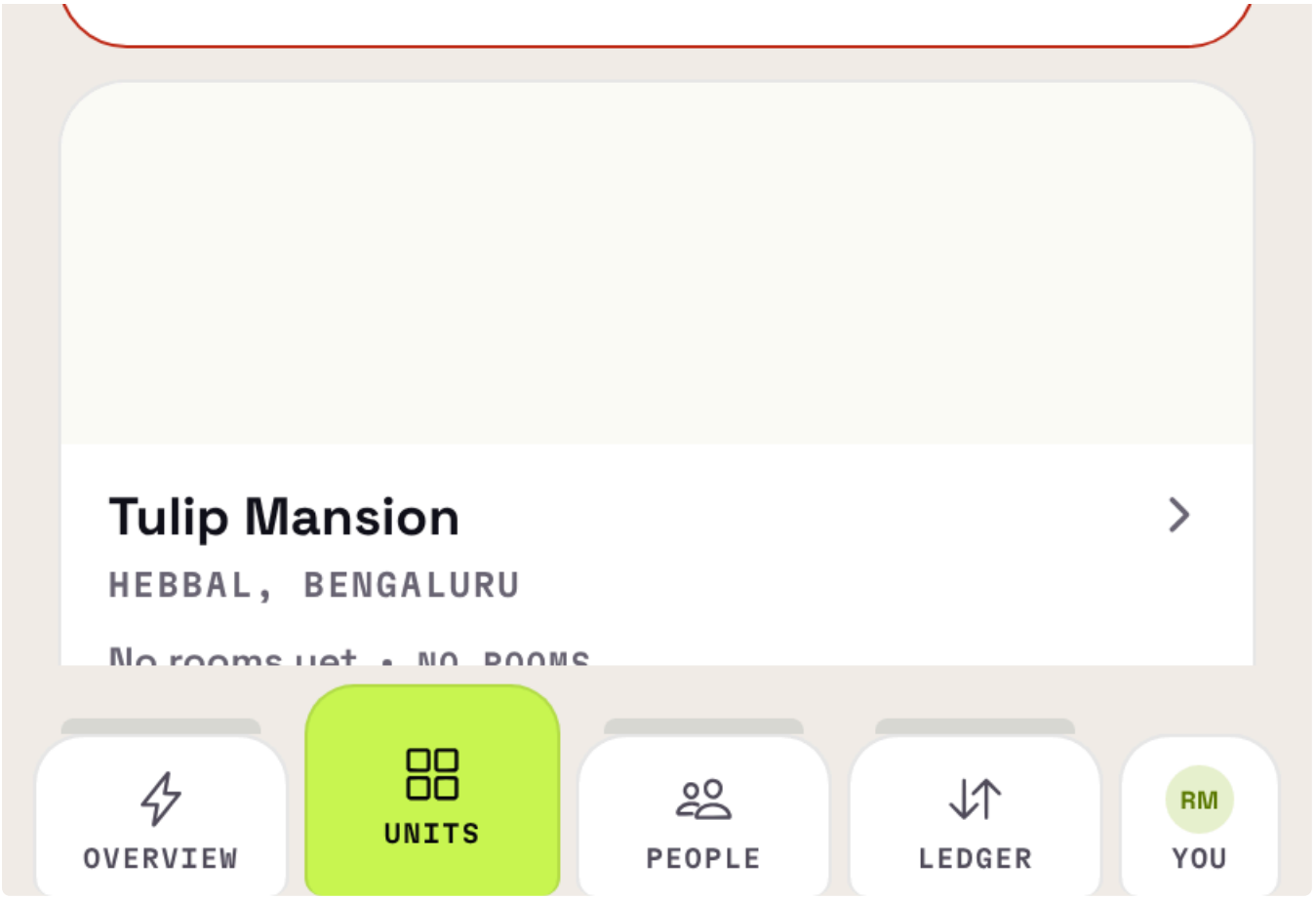


Each row carries the tenant's property and room, their share of the rent, a payment-behaviour score and a due-state chip: `IN 7D`, `19D LATE`, or a leaving date once a tenant has given notice. The filter chips carry counts, so the spread is visible before any of them is tapped.

`UNASSIGNED` was not bolted on afterwards as an edge case. A tenant is allowed to exist without a room, and that state is reachable from the roster instead of being tucked away somewhere. The end-to-end suite checks that such a row renders without a broken room label.

3.3 Rooms and properties


Property, unit or tenant
2
Properties & units
3 properties · 6 rooms · 0 vacant
+ Add property
Palm Heights
BANASHANKARI, BENGALURU
Every room filled • 2 ROOMS
2 / 2 FULL
₹7,250 due · 3B
Silver Oak Nest
RAJAJINAGAR, BENGALURU
Every room filled • 4 ROOMS
4 / 4 FULL



Properties hold rooms and rooms hold tenants, each room with its own rent, its capacity, and a split across occupants that can be equal or set by hand. A room's status follows its occupancy on its own. Notification preferences are per room, with email, SMS and WhatsApp switched independently, because a landlord rarely wants the same channel across a whole portfolio.

3.4 Ledger

Ledger

Every rupee in and out · ALL PROPERTIES

NET — OCTOBER

₹48,500

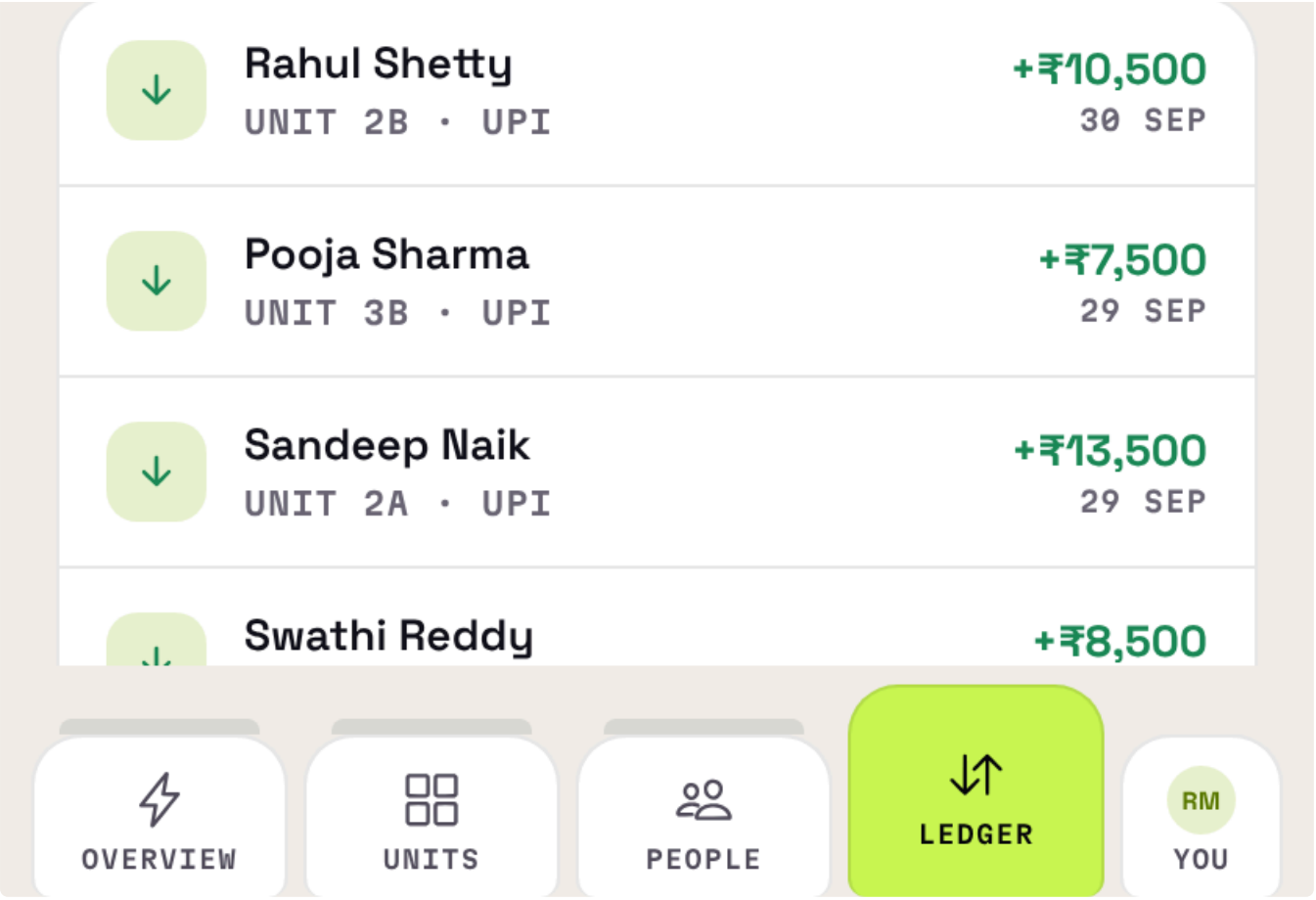
IN ₹48,500
OUT ₹0

Search tenant, unit or UTR

OCTOBER 2026 +₹48,500

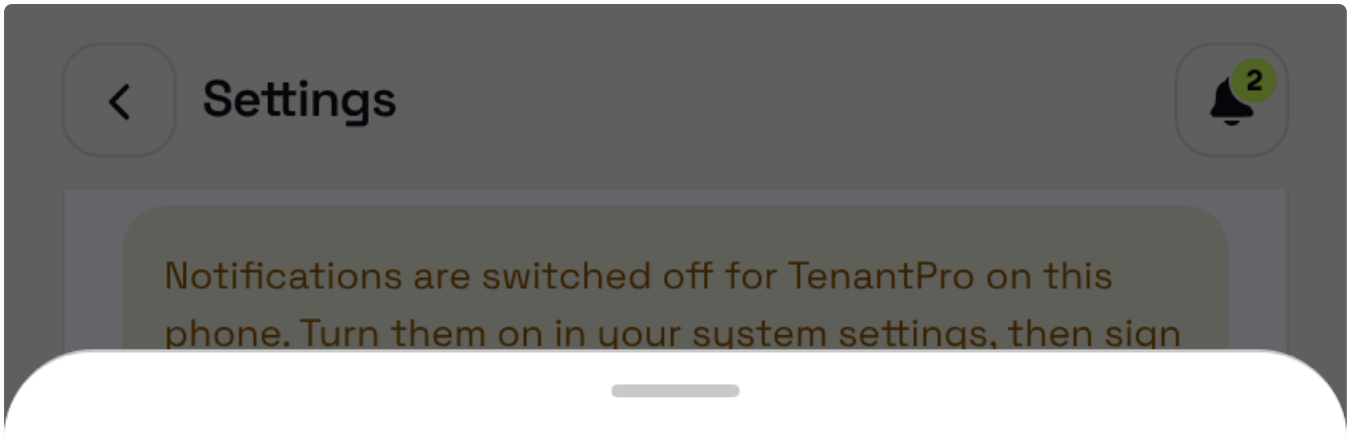
Akash Shetty UNIT 2C · UPI	+₹16,500 4 OCT
Suresh Prabhu UNIT 3B · CASH	+₹7,250 2 OCT
Deepa Shetty UNIT 2A · BANK TRANSFER	+₹8,000 2 OCT
Ritu Menon UNIT 2A · CASH	+₹8,500 1 OCT
Kavya Prabhu UNIT 1D · UPI	+₹8,250 1 OCT

SEPTEMBER 2026 +₹72,750



Every confirmed payment, searchable and narrowable to a single property. Each payment stores the due date it settled, captured at the moment of declaration. Without that field, "was this on time?" becomes unanswerable after the fact: confirming a payment moves the tenant's next-due date forward, and that overwrites the only evidence there was. Leaving it out is why the behaviour score once reported "None missed" for every tenant on the books.

3.5 Connecting a payment gateway



Online payments

Tenants can pay rent by card, netbanking or any UPI app, and it is confirmed automatically — you will not have to tap Confirm.

WHAT IT COSTS

The account is yours, so rent lands in it directly. Razorpay takes its usual fee (about 2% + GST) from each payment — this app charges you nothing, and your tenants pay only the rent.

Your UPI details stay exactly as they are. Tenants keep the free UPI option either way.

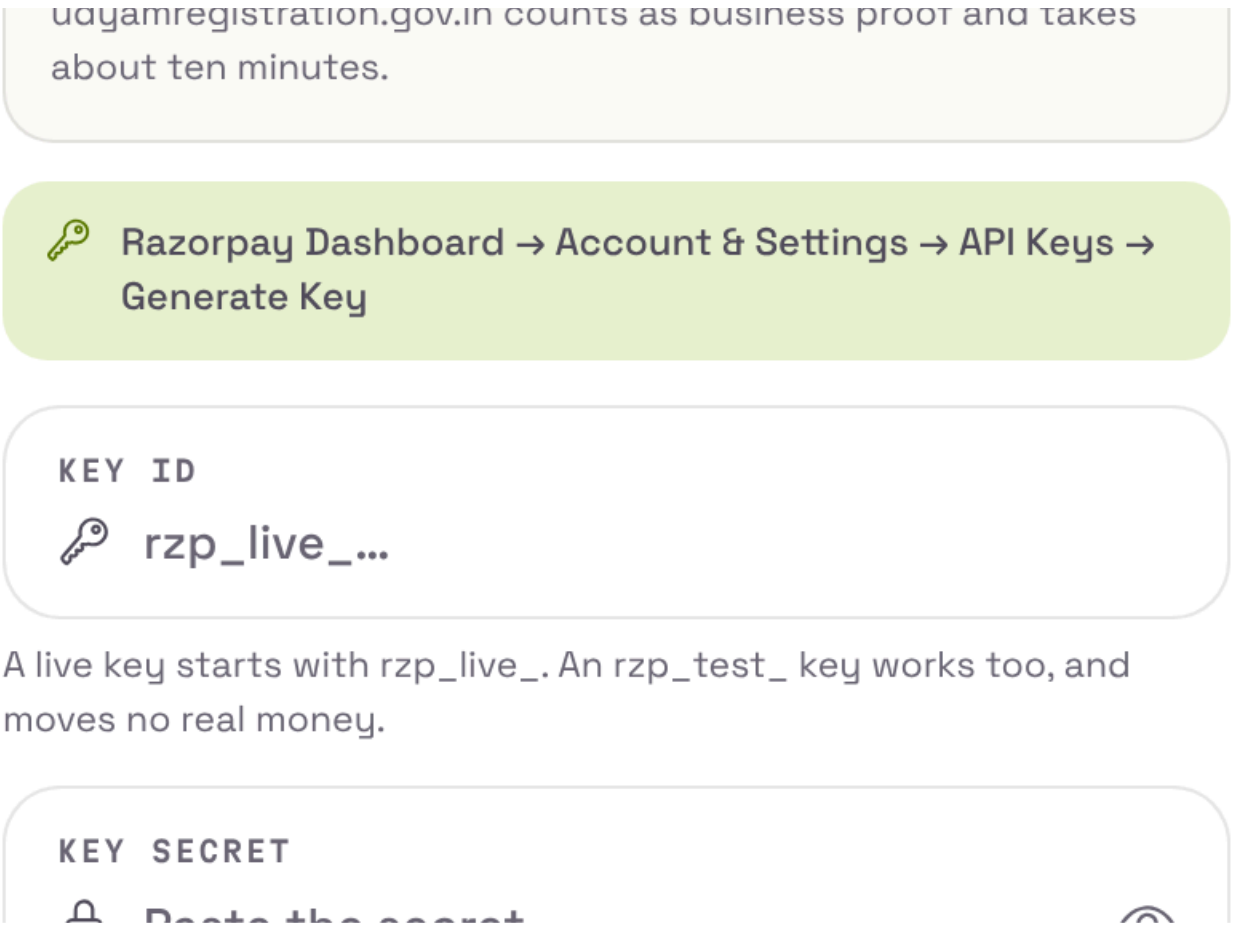
BEFORE YOU START

Razorpay will ask for your PAN, a bank account, and one business proof.

Live keys usually take 2–5 working days — KYC first, then a separate check of the website you give them.

✓ Test keys work straight away. Your tenants can keep paying by UPI the whole time.

No GST? A free Udyam (MSME) registration at



The sheet puts the price right next to the offer. Describing one without the other is how somebody switches on a 2% fee without noticing. It also spells out what the gateway is going to ask for before the landlord sets off: PAN, a bank account, one business proof, and two to five working days. It adds that test keys work straight away and that tenants can carry on paying by UPI the whole time. A landlord who expects five minutes and runs into a week concludes the application is broken.

Credentials are checked against the gateway *before* anything is stored, so a mistyped secret gets caught while the landlord still has the form in front of them. What is stored is sealed with AES-256-GCM, and it is never returned to any client.

3.6 Other implemented landlord features

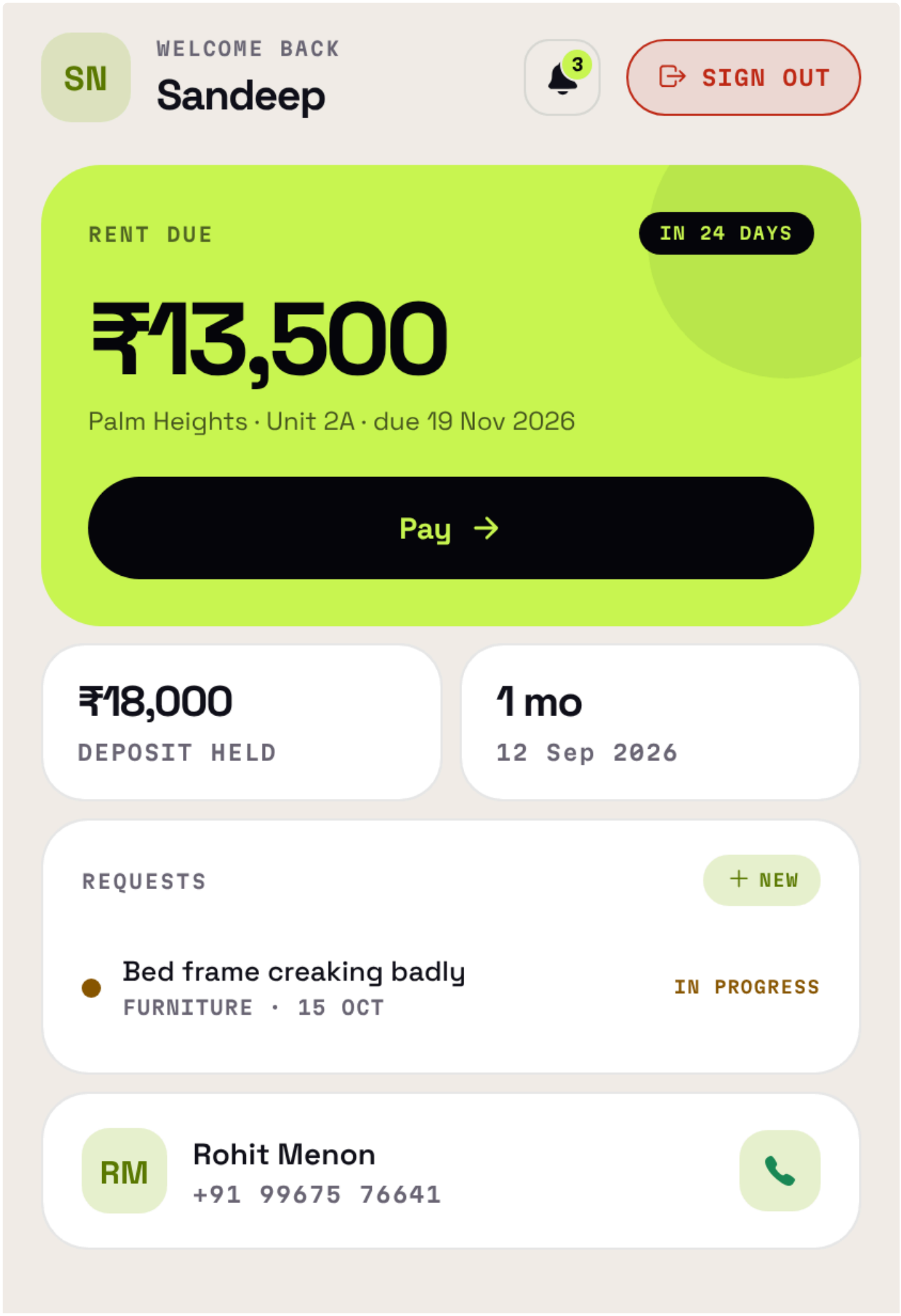
- **Join requests.** A tenant asks to join a property using a code, and the landlord accepts or declines, assigning a room at the same time if they want to.
- **Maintenance threads.** A conversation per request, shared with the tenant, where status changes are recorded as events in the thread itself.
- **ID documents.** Proofs uploaded by the tenant. For tenants who have moved out, the preview is blurred and served under a short-lived token.
- **Payment confirmation queue.** Declared payments with enough context around them to decide, confirmed or rejected with a reason the tenant gets to read.

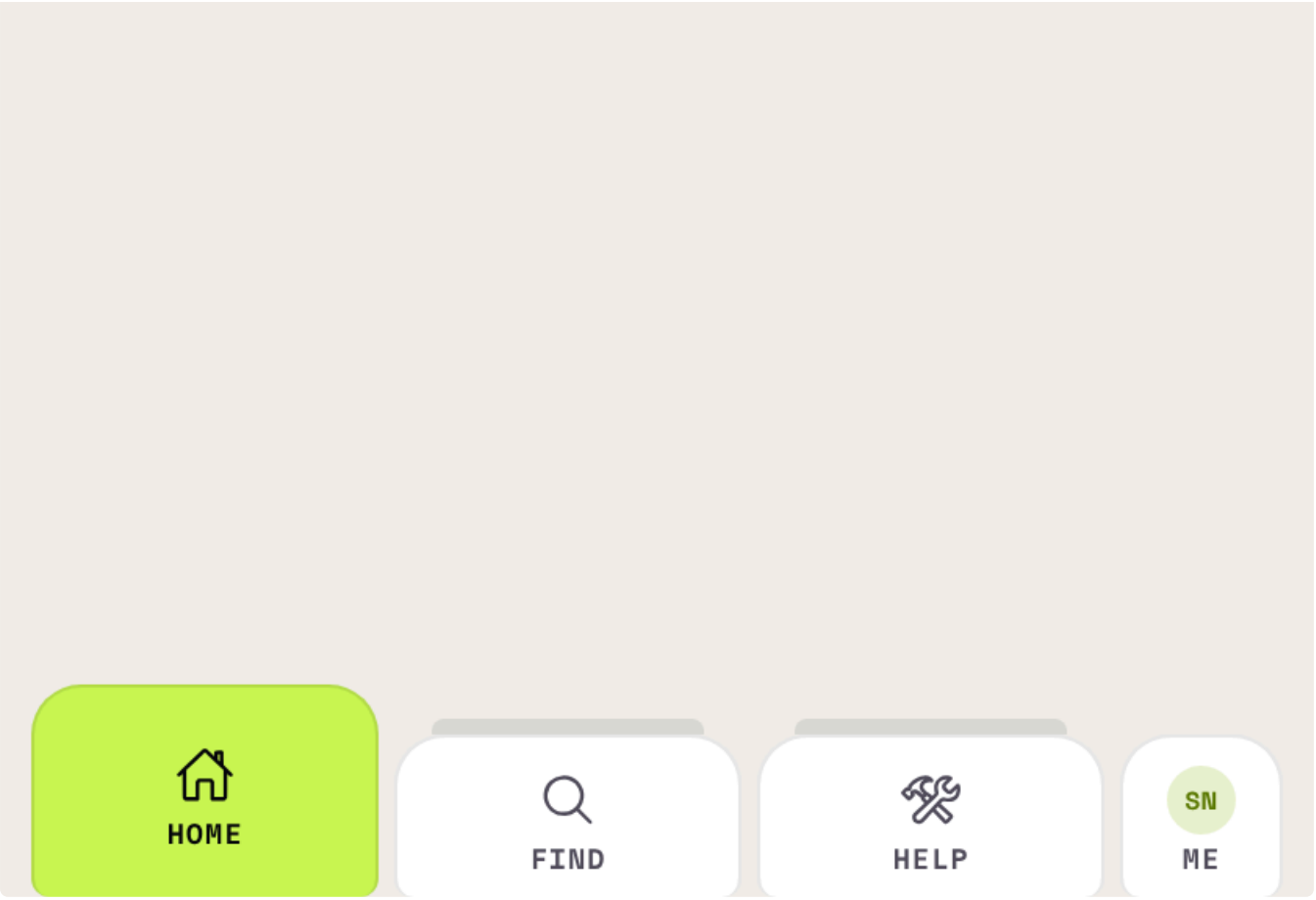
- **Notice periods.** A tenant's leave date, worked out on the server and shown to both sides.
- **Daily rent reminders.** An 8am job across email, push, SMS and WhatsApp, with every channel phrasing the same day the same way.
- **Google sign-in**, profile management with contact verification, and a reset for the demo account.

4. Tenant-side implementation

The tenant portal is not a cut-down version of the landlord app. It answers a different question: what do I owe, and has anyone acknowledged it? Getting in means a separate account type, with its own table and its own token role.

4.1 Home

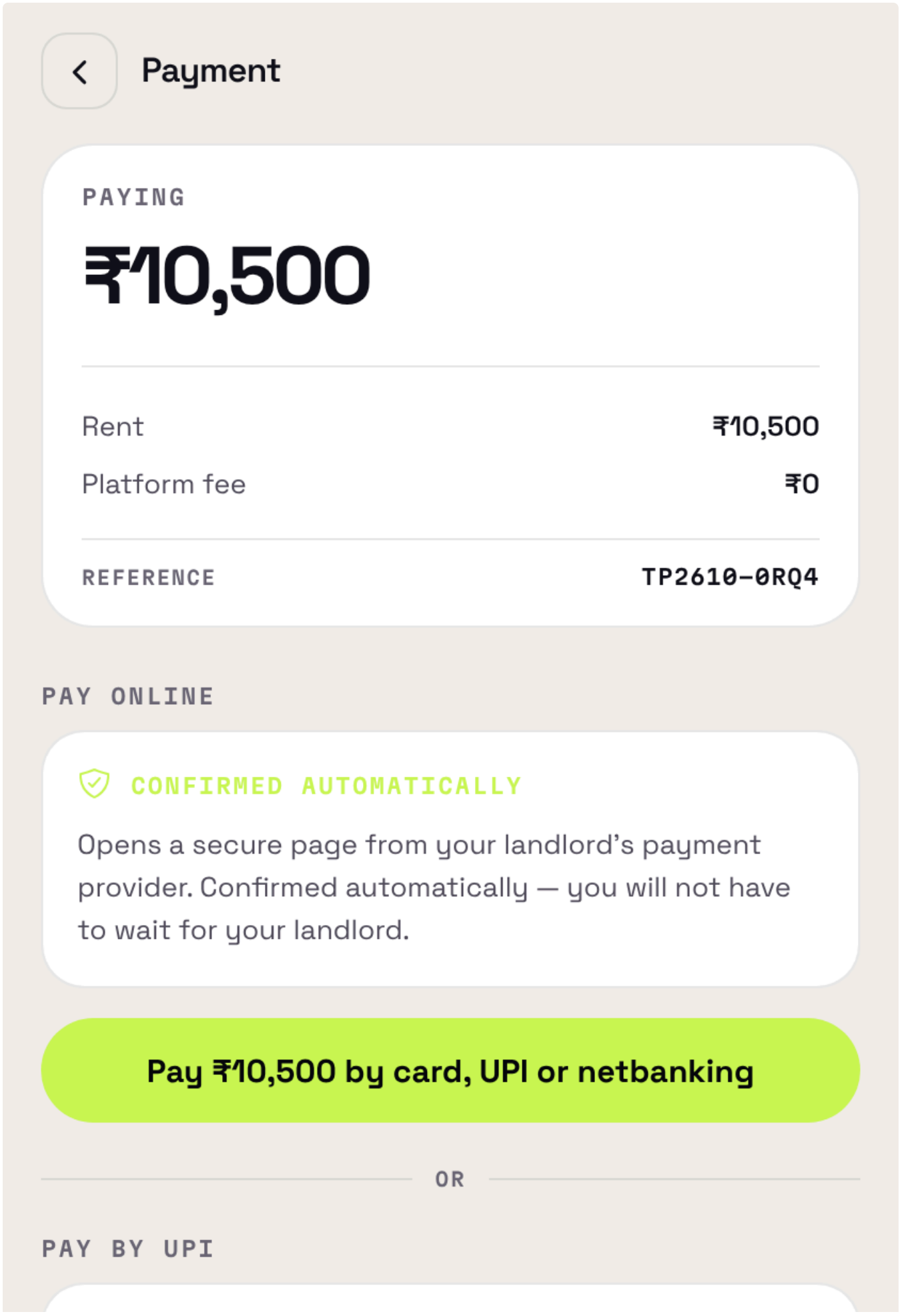

SN
WELCOME BACK
Sandeep
3
SIGN OUT
RENT DUE
IN 24 DAYS
₹13,500
Palm Heights · Unit 2A · due 19 Nov 2026
Pay →
₹18,000
DEPOSIT HELD
1 mo
12 Sep 2026
REQUESTS
+ NEW
Bed frame creaking badly
FURNITURE · 15 OCT
IN PROGRESS
RM
Rohit Menon
+91 99675 76641

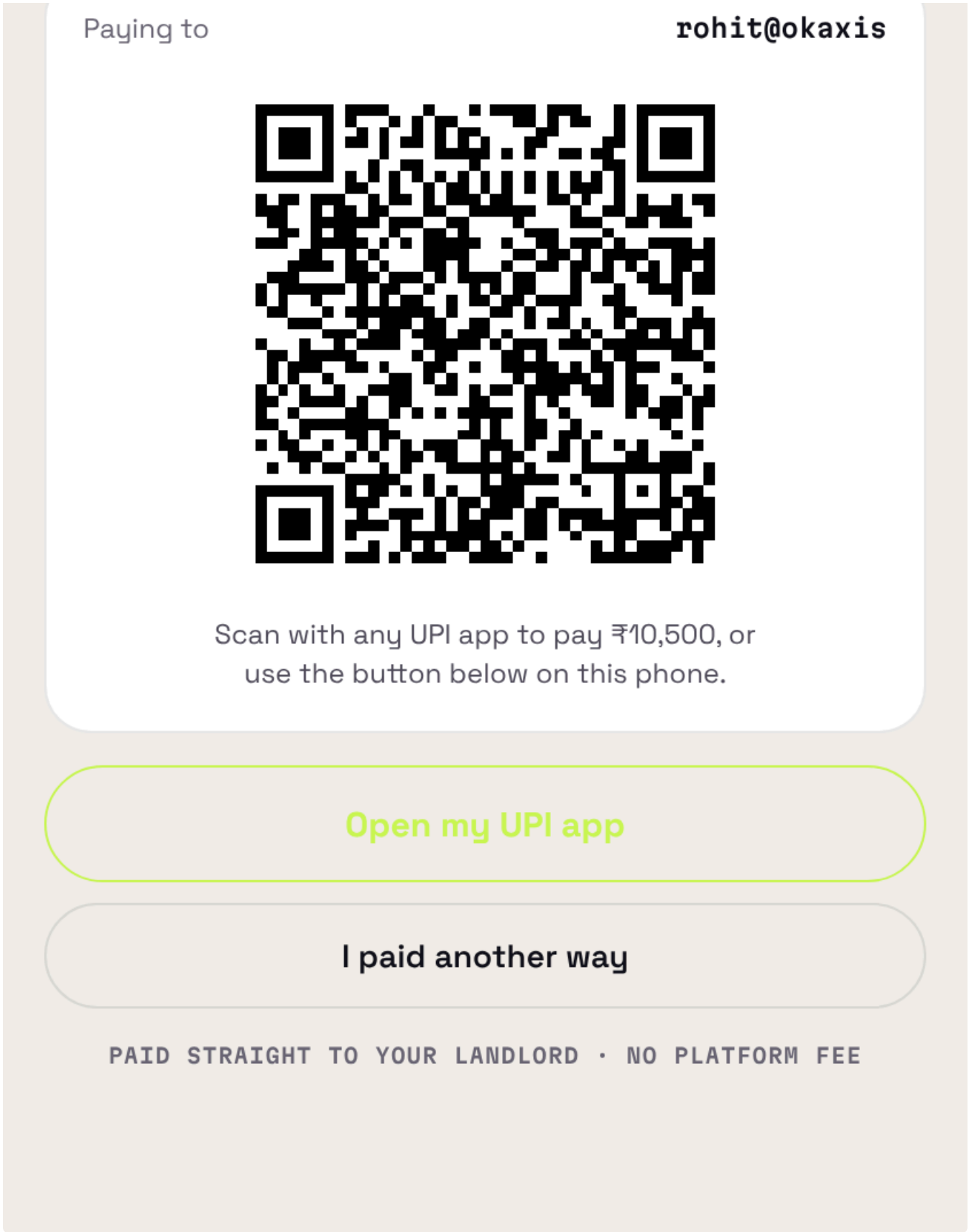


The rent card carries the amount, the property and room, the exact due date and a countdown to it. Deposit held and tenure sit underneath. Both are things tenants routinely cannot remember and have no other record of.

The bell works off the same payload the screen renders, not a separate unread count that could drift away from it. What shows up there: rent falling due, a landlord's reply on a maintenance thread, a payment confirmed or rejected, and the outcome of a join request. An `alerts_seen_at` timestamp tells an alert that is *news* apart from one that is merely still true. The server stamps it, so a phone with the wrong clock cannot silence its own alerts.

4.2 Paying rent


Payment
PAYING
₹10,500
Rent
₹10,500
Platform fee
₹0
REFERENCE
TP2610-0RQ4
PAY ONLINE
CONFIRMED AUTOMATICALLY
Opens a secure page from your landlord's payment provider. Confirmed automatically — you will not have to wait for your landlord.
Pay ₹10,500 by card, UPI or netbanking
OR
PAY BY UPI



The screen shows what is owed, a reference and ₹0 as the platform fee. On both routes that figure is literally true.

Where the landlord has connected a gateway, **Pay by card, UPI or netbanking** comes first, since it is the only route that confirms itself. Below an `OR` divider sits the UPI route: the landlord's VPA, the amount and the reference, encoded as a QR that works when scanned from another phone, with a hand-off to the tenant's own UPI app for people paying on this one.

The two buttons look different on purpose. Two filled primary buttons amount to two primary actions, which is the same as having none. So the UPI button is outlined when something louder sits above it, and filled when it is the only thing on offer.

Coming back from a gateway page, the button reads **Check my payment** rather than "Yes, I paid". On that route the answer is not the tenant's to give.

4.3 Profile and payment history

My profile

SN

Sandeep

UNIT 2A · Studio

12 Sep 2026

1 mo
WITH YOU

₹13,500
RENT SHARE

₹18,000
DEPOSIT

+26
CREDIT

YOUR CREDIT SCORE

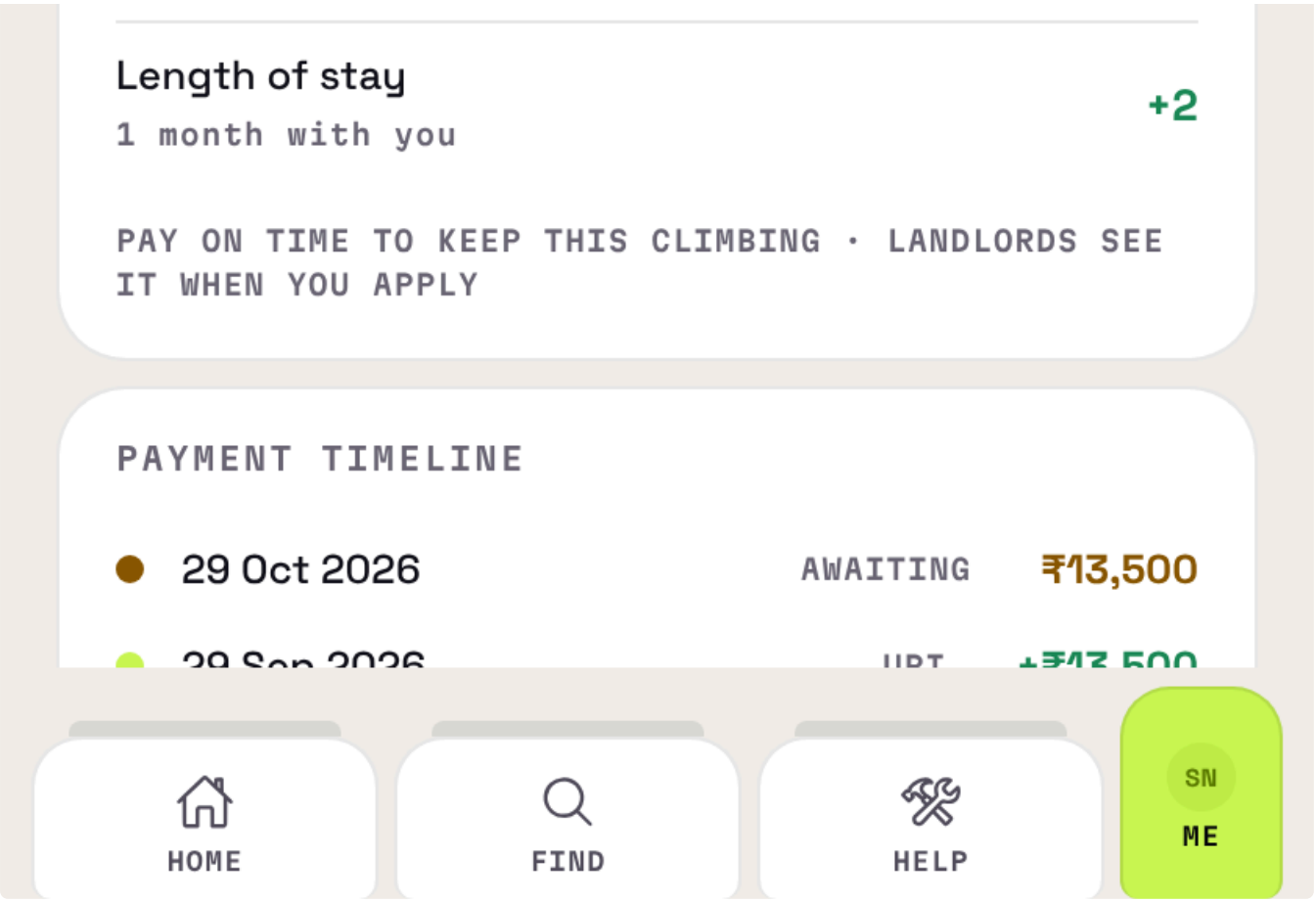
+26 Building

−100 NEGATIVE POSITIVE +100

Paid on time
4 payments on time
+24

Paid late
Never late
0

Currently overdue
Nothing pending
0



The behaviour score runs through the same code that produces the landlord's copy of it. Sharing the code is the whole point. A tenant looking at a different number from the one their landlord sees cannot argue with either of them. The factors are listed out instead of being boiled down to a dial, and where there is too little history to say anything, the screen says so. The alternative is a needle parked at zero, which reads like a verdict the tenant has earned.

The payment timeline is built from real rows. A claim still waiting on the landlord shows an amber `AWAITING`, not the green reserved for money that has actually arrived. Show the two alike and a tenant will believe their month is clear when it is not.

4.4 Maintenance

Help

+ Raise a ticket

Report an issue and follow it through.

Bed frame creaking badly

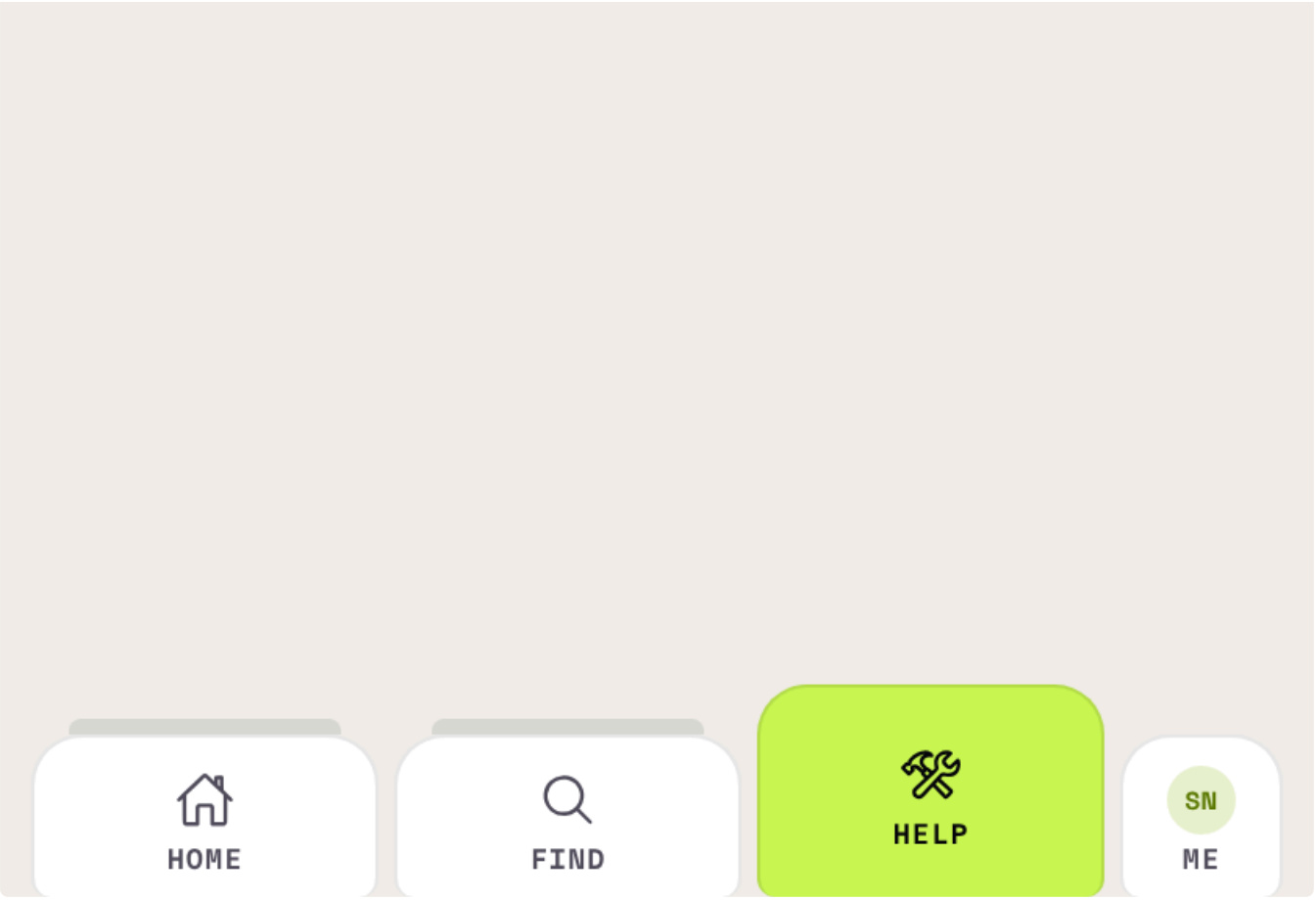
FURNITURE · 15 OCT

IN PROGRESS

RM

Rohit Menon

+91 99675 76641



A tenant raises a request with a category, a priority and a photo if they have one, then follows it in a thread shared with the landlord. Status changes land in that thread as events, in order with whatever was said around them.

4.5 Other implemented tenant features

- **Guest accounts.** A tenant with no email, identified by a code, with a stay window attached. Expiry is enforced on the server, and the lock-out message offers the one action that resolves it.
- **Claiming an account.** A guest turns into a full account with a name, email and password.
- **Joining a property.** By scanning or typing an invite code, with the property shown before the request goes anywhere.
- **Receipts.** The full payment history, with status, reference and the rejection reason wherever one was given.
- **Giving notice.** The real dates are worked out and shown before anything is written, and the notice can be withdrawn afterwards.
- **ID documents.** Upload and removal, with a summary the landlord sees while deciding on a join request.
- **Directions.** To the tenant's own property, where the landlord has pinned it.

5. The payment and settlement model

There are three ways to move money between two parties on a platform. TenantPro uses the third, and that was a choice rather than an accident.

Model	Who holds the funds	Authorisation required	Example
Aggregator	the platform, in a ring-fenced escrow at a scheduled commercial bank	RBI payment-aggregator licence	Uber, Ola
Split settlement	the gateway, settling to linked sub-merchant accounts	held by the gateway, not the platform	Razorpay Route
Direct	nobody — payer pays payee	none	Namma Yatri, **TenantPro**

RBI rules do not let an aggregator keep customer funds in its own account. The money has to sit in a segregated escrow, where it earns the platform no interest and has to be settled out by a deadline. Getting that authorisation in the first place means clearing a net-worth floor in the tens of crores. TenantPro sidesteps the whole question by never receiving the money.

5.1 Two payment routes, both direct

UPI. The tenant's screen carries the landlord's own VPA, the exact amount and a generated reference, encoded both as a QR and as a `upi://` intent. The tenant pays from whichever UPI app they already use. It costs everybody nothing.

Gateway. A landlord can connect their own Razorpay merchant account. The server then creates a Payment Link using *their* credentials, the tenant pays on Razorpay's hosted page, and the money settles to that landlord. Razorpay charges its fee to them, through their own account. The platform bills nothing, because it is party to nothing.

Payment Links were picked over Razorpay's React Native SDK for one reason: the SDK is a native module. A link is plain REST over HTTPS, which means the payment feature ships over the air like everything else.

5.2 The confirmation model

A payment starts out as a `Declared` row. That is a claim, and it clears nothing on its own. It becomes real when a second party vouches for it:

- the **landlord** taps Confirm, or

- the **gateway** is asked outright, server-to-server with the landlord's credentials, and says the link was paid.

Both paths end up in the same function, `settlePayment`, which flips the status and moves the tenant's next due date forward inside one transaction. Write it twice and the two copies will eventually disagree about when a month has cleared, so it is written once.

The gateway gets polled rather than trusted to call back. A webhook would mean every landlord configuring a URL and a secret in a dashboard, and most of them would never get to the end of that. The signed-webhook path does exist and is still supported; it is just the secondary route.

6. Authentication and session management

The session design went through three versions. The sequence is worth following, because each of the first two fixed something real and created the next problem on the way.

Stage	Access token	Can be revoked	Failure it solved	Failure it left
Original	7 days, fixed	no	—	weekly logout for every user
Sliding	30 days, renewed on use	no	the weekly logout	nothing could be cancelled
Current	2 hours + refresh token	yes	revocation	instant revocation on the access path

6.1 Why a longer token was not the fix

Raising the lifetime from 7 days to 30 did not stop people being logged out. It made it happen monthly. A fixed lifetime counts from the last *sign-in*, which tells you nothing about whether somebody is still using the app. Open it every morning and you were thrown out on day 30, the same as somebody who had not opened it since February.

So the server re-issues the access token on any authenticated request once it is past half its life, handing it back on an `X-Session-Token` header that the client picks up. The new token's lifetime comes from the old one's `exp - iat` rather than from a constant. A guest session therefore keeps its own length, and older tokens keep theirs.

6.2 Refresh tokens and revocation

An access token is a signature and nothing more. The server consults no list when it verifies one, which is what makes it fast, and also meant there was no way to withdraw it. A stolen phone, a tenant who had moved out, a password change: none of them ended a session.

A session is now a row as well, holding a refresh token that is stored as a SHA-256 hash rather than as the token itself. Three properties of that design are worth naming:

1. **Rotation.** Spending a refresh token gets you a replacement. A sixty-day secret that crosses the network on every reconnect is a long-lived credential. One that works once is not.
2. **Reuse detection.** Rotation means the honest client is always holding the newest token, so a replay of a spent one says a copy is sitting somewhere else. Every session on the account is revoked. Getting that wrong in one direction costs somebody a sign-in; getting it wrong in the other costs them the account.
3. **Revocation on password reset.** Resetting a password now ends every other session. Users had always assumed it did.

On the refresh path, revocation is immediate. On the access path it lags by at most the access token's two-hour life. Closing that last gap would mean a database read on every authenticated request, which is a cost almost nobody pays. The gain here is two hours where it used to be thirty days.

6.3 Client-side failure handling

If this design is going to fail badly, it will fail on the client. A 401 triggers one refresh and one replay. Only a 401 *from the refresh endpoint itself* ends the session; a 500, a timeout or a dead connection does not. The server answers 500 rather than 401 on its own faults deliberately, so that the client can tell the two apart.

The nastiest failure mode was concurrency. On a cold start the app fires eight requests at once, and if they all fail on the same expired token, the naive version issues eight refreshes. Seven of those replay a token the first one has already spent, the server reads that as theft, and every session ends. A feature built to prevent lockouts would have been causing them at scale. One shared in-flight promise is what stops it, and there is an assertion on that behaviour.

7. Verification

386 assertions across six suites stand behind the build. None of them uses a test framework. Each suite is a plain Node script that exits non-zero when something fails.

Suite	What it drives	Isolation	Assertions
`secretbox`	AES-256-GCM sealing of gateway credentials	pure	24
`session`	token renewal, through the real middleware and the real interceptor	stubbed db	41
`gateway`	gateway controllers	stubbed db and `fetch`	72
`vm.gateway`	the real `deriveVm`, react-native stubbed	stubbed native	52
`refresh.client`	the real axios interceptor: refresh, retry, and when not to sign out	stub adapter	29
`refresh.e2e`	refresh tokens, reading the database directly	live server + MySQL	42
`api.e2e`	all 87 endpoints	live server + MySQL	126
`ui.e2e`	the compiled application in a browser engine	live stack + Chromium	71

7.1 Seeded fixtures

A deterministic generator (`scripts/seed.js`) builds a portfolio that deliberately contains the rows which break screens and never turn up when you are looking for them: a tenant with no room, a guest with no email, a guest whose stay has *already* ended, rent overdue and due today and upcoming and never set at all, a payment declared and undecided, another rejected with a reason, maintenance threads where the landlord spoke last and others where the tenant did, a tenant who has given notice, and a property with no rooms in it. The PRNG is seeded, so any failing run can be reproduced.

7.2 What the browser suite does and does not cover

The UI suite runs the same JavaScript the phone runs: the same view-model, the same screens, the same API client and interceptors, laid out by `react-native-web` , which is what an Expo web export produces. What it cannot reach is anything sitting behind a native module, so camera, image picker, push delivery and the UPI intent hand-off are all out of scope. For those it prints `SKIP` instead of a pass, so a green run never claims coverage it does not have.

7.3 Defects found by the end-to-end runs

Driving every endpoint against a real database, and the application against a real browser, turned up five defects the unit tests had missed.

Defect	Severity	Status
Two of three endpoints accepting a `unit_id` checked ownership; the third did not, allowing one landlord's tenant to be filed into another's room	cross-account read	fixed — one shared check
A wrong password rendered "Your session expired", overriding the server's own `WRONG_PASSWORD` message	user-facing	fixed
Five required-field omissions produced HTTP 500 with a raw SQL error instead of a 400	robustness	fixed
Renaming a property without its address crashed: `NOT NULL` column, coerced on insert but not on update	robustness	fixed
A network failure reaching the gateway was reported as a rejected credential	user-facing	fixed — transport failures now distinguished

Two more defects turned up **in the test harnesses themselves**, which is worth putting on the record. The first browser run never got past the permissions primer, and because its assertions were phrased as the *absence* of a string the primer also happened to lack, half the suite passed while testing nothing at all. Separately, the failure reporter crashed on a failure whose context was `undefined` and took down the run it was supposed to be reporting. The lessons are to assert on what should be there rather than on what should not, and to keep the failure reporter the dullest code in the file.

8. Limitations

Set out plainly, since each of these follows from a design decision rather than from an oversight.

The application cannot observe money. It never sees a bank balance, so a UPI payment stays a claim until the landlord confirms it. For landlords who connect a gateway, that goes away. For everyone else the confirmation step is structural.

Revocation lags by up to two hours on the access path. Closing the gap completely would mean a database lookup on every authenticated request.

Reuse detection can produce a false positive. If a client stores a new access token but fails to store the refresh token that came with it, the next request presents a spent token and every session gets revoked. The client-side write is transactional, but a device that loses its storage halfway through would still land here.

SMS is unconfigured. Phone verification needs a DLT registration with an Indian telecom operator, and that has not been done. Email and push are both live.

iOS has never been built. `ios.bundleIdentifier` is still unset, and the project has only ever produced Android artifacts.

Gateway onboarding is heavy. A landlord connecting Razorpay has to get through their KYC and a separate website verification, which realistically takes two to five working days. The in-app sheet now says so before they start, but it is still the biggest piece of friction in the product.

No automated regression gate. The suites exist and they pass, but nothing runs them on push. Verification is a manual step somebody has to decide to take.

9. Future scope

9.1 Identified and specified

Reply notifications on maintenance threads. When a tenant replies to a repair request, nobody is told: `createRequestMessage` makes no `notify` call, and the push rules have no owner-facing kind for a reply. The landlord-to-tenant direction already works.

A transaction around `recordPayment`. The landlord's own entry path inserts the payment and updates the due date in two separate statements. `settlePayment`, on the tenant-facing side, is already transactional. This path is not, and a failure between the two statements leaves a payment recorded against a date that never moved.

An automated test gate. The suites are reproducible and quick enough to run on every push. Wiring them into CI is configuration work rather than development work.

iOS. A bundle identifier and a first build.

9.2 Larger directions

Split settlement, once there are landlords other than the author. Razorpay Route would let landlords onboard as linked sub-merchants from inside the application, with the gateway doing their KYC. That would take away the onboarding friction described in Section 8. The cost is that funds would then pass through a platform account, which is a

change of category and not of configuration, and the 2025 RBI directions restrict split settlement in ways that call for professional advice rather than inference. Worth coming back to when the user base justifies it. Not before.

Monetisation without entering the money path. The Namma Yatri precedent applies here almost directly. A flat per-landlord subscription keeps the operator out of the payment flow altogether, whereas a percentage of each transaction starts to look structurally like aggregation. There is a related legal nuance: models like this rest on selling access to software rather than on setting the price. In TenantPro the landlord sets the rent, and that is a position worth holding on to deliberately.

Deliverability. Transactional mail goes out from a Gmail address through Brevo's HTTPS API. A verified sending domain with SPF, DKIM and DMARC would improve inbox placement a good deal, for somewhere around ₹1,000 a year.

SMS and DLT. Phone verification needs an Indian DLT registration before any operator will carry template traffic.

Landlord announcements. A one-to-many message to everyone in a property ("water off Tuesday") was specified during the notification work and then never built. The alert infrastructure that would carry it is already there.

10. References

The code figures, endpoint counts and assertion totals in this report were measured from the repository at commit `f065250` and from live test runs. None of them are estimates.

Regulatory and payment-model sources

- Nodal and escrow accounts under RBI rules — how aggregators are required to hold customer funds.
- RBI rewrites the payment aggregator rulebook — commentary on the Regulation of Payment Aggregators Directions, 2025.
- Razorpay Route — split settlement and linked accounts.
- Razorpay API keys — key generation, and the website verification required for live mode.
- Razorpay KYC documents by business type.
- Namma Yatri's zero-commission model — the direct-payment precedent discussed in Section 5.

Note on secondary sources. Several of the regulatory points above rest on consultancy and industry commentary rather than on the RBI's own text. Where this report leans on them, particularly for the net-worth thresholds and the restrictions on split settlement,

the claims should be checked against the Directions themselves before anybody makes a structuring decision.